

Отчёт по индивидуальному заданию 1

Вариант 20 Автогонки

Последовательная консольная программа на C для Linux

Исполнитель: Куслин Глеб Витальевич

Группа: БПИ252

В работе реализована последовательная модель автогонок на C для Linux. Программа отображает движение по клеткам, выполняет правила раундов, обрабатывает блокировки, сход и финиш, формирует результаты машин и команд. Ниже описаны принятая модель и результаты проверки текущей версии.

1 Условие задачи

Разработать консольное приложение, моделирующее гонку по кольцевой клеточной трассе и стратегии водителей. Базовая команда состоит из двух автомобилей. Машины стартуют на внешней дорожке в порядке жеребьёвки. Перед каждым раундом участники упорядочиваются по пройденной дистанции; при равенстве преимущество имеет внутренняя дорожка.

Лидеру разрешено потратить от одной до четырёх единиц движения. Для второй машины предел равен фактически потраченным единицам лидера плюс два, для остальных — фактической стоимости хода предыдущей машины плюс один. Движение вперёд стоит одну единицу, переход внутрь — одну, наружу — три. Запрещены движение назад, диагонали, вход в занятую клетку и повторное посещение клетки за один ход. Невозможность сделать допустимый переход означает нулевой ход и сход после окончания раунда.

Гонка завершается после финиша или схода всех машин либо по достижении предела раундов. Необходимо сформировать итоговую классификацию машин и команд.

Источники: `i-tasks.pdf`, страницы 40–42, и `i-tasks-req.pdf`, требования к последовательной программе. Подробные правила исходной игры приведены в [публикации правил В. Пудова](#).

2 Принятая модель

Используется кольцо с фиксированной шириной. Каждая дорожка содержит L клеток, соответствующих одним и тем же продольным секторам. Клетка задаётся парой «дорожка, сектор». Последний сектор соединён с первым. Переход между дорожками сохраняет сектор. Все дорожки содержат одинаковое число клеток; это выбранная модель трассы, а не точная копия поворотов на рисунке условия.

Дорожка 1 — внутренняя, последняя — внешняя. Стартовая и финишная линия находится в клетке 1 каждой дорожки. Стартовая колонна расположена перед линией: у лидера продвижение -1, у следующей машины -2 и так далее. На дорожке должно быть хотя бы на одну клетку больше, чем машин.

При первом пересечении стартовой линии продвижение становится нулевым; это начало отсчёта кругов. Каждый последующий переход вперёд увеличивает продвижение на единицу. Боковой переход не увеличивает его. Круг завершён при достижении L , $2L$ и так далее. Финиш наступает при продвижении $K \cdot L$, где K — заданное число кругов. Машина останавливается точно на линии.

Жеребьёвка определяет порядок команд. Сначала расставляются первые машины всех команд, затем следующие машины в том же порядке. Такой порядок соответствует

разделу «Машины на старте» исходных правил; число машин в команде можно изменить параметром программы.

Круговые премии и дополнительные правила снятия отставших отключены. Нулевой ход определяется только отсутствием допустимого первого перехода, а не неудачей случайного выбора. Окончание короткого непутого маршрута не считается сходом.

Финишировавшая машина остаётся препятствием до окончания раунда, как указано в разделе «Как финишировать?» исходных правил. Аналогично до конца раунда остаётся на трассе машина с нулевым ходом. После этого обе удаляются из массива занятости и больше не ходят.

3 Сущности и структура программы

Все события имитируются последовательно в одном процессе. Автомобиль и судья являются логическими участниками модели; дополнительные процессы Linux и потоки не создаются.

Сущность	Представление	Назначение
Конфигурация	Config	Параметры гонки, стратегия, seed и журнал
Автомобиль	Car	Номер, команда, клетка, продвижение, состояние, место финиша
Трасса	track[lane] [position]	0 — свободная клетка, иначе номер машины
Маршрут	Route	Последовательность направлений и их суммарная стоимость
Судья	run_round, finish_round, create_results	Очередность, пределы ходов, сход и классификация
Гонка	Race	Общее состояние всех сущностей

Внутренние индексы дорожек и клеток начинаются с нуля; в выводе — с единицы. Отдельно хранятся продвижение вдоль трассы и стоимость маршрута: боковой переход расходует запас движения, но не меняет пройденную дистанцию.

4 Стратегии и алгоритм

Стратегия greedy выбирает движение вперёд, если оно допустимо; иначе переход внутрь, затем наружу. Стратегия random выбирает случайный переход из допустимых. Выбор повторяется до исчерпания запаса, отсутствия следующего перехода или финиша. Глобальный поиск лучшего маршрута не выполняется.

При построении маршрута исходная клетка и все посещённые клетки отмечаются в отдельном массиве. Для каждого перехода проверяются границы, занятость, оставшийся запас движения и отсутствие повторного посещения. Остальные машины во время хода неподвижны. Построенный маршрут затем выполняется с обновлением массива занятости.

В начале раунда формируется отдельный массив индексов участвующих машин. Он сортируется по продвижению, внутренней дорожке и номеру машины. Массив не меняется до конца раунда, даже если произошёл обгон или финиш. После каждого хода его фактическая стоимость задаёт предел следующего. После последнего участника судья снимает машины с нулевым ходом и финишировавшие, пересчитывает порядок следующего раунда и отображает состояние трассы.

Порядок формирования результатов:

1. Финишировавшие — по хронологическому порядку финиша.
2. Остальные, включая сошедших и не завершивших гонку, — по достигнутому продвижению. При равенстве выше внутренняя дорожка, затем меньший номер.
3. Команды — по месту своей лучшей машины в общей классификации.

Это явно выбранные правила конфигурации. Сошедшая машина сохраняет продвижение на момент схода. При лимите раундов и прерывании оставшиеся участники получают состояние «не завершила», а не «сход».

Число машин ограничено 32. Предел хода не превосходит 36: у первой машины 4, у второй не более 6, затем увеличение не более чем на 1. Поэтому массив из 36 шагов достаточен для любого допустимого хода этой модели. Построение маршрута рассматривает не более трёх переходов на каждом шаге. Сортировка использует вставки; для не более 32 машин её стоимости достаточно.

5 Интерфейс и завершение

Сборка и обычный запуск:

```
make
./race
```

Полный список параметров доступен через `./race --help`. По умолчанию: 3 команды по 2 машины, 4 дорожки по 24 клетки, 2 круга, максимум 100 раундов, `seed=1`, стратегия `greedy`. Изменяемые параметры можно задать аргументами командной строки. Геометрия кольца, жеребьёвка старта, отключённые премии, правила схода и классификации заданы непосредственно в программе; такой способ задания параметров допускается требованиями на странице 2. Числа и взаимная совместимость параметров проверяются до создания гонки.

Параметр	По умолчанию	Допустимые значения
<code>--teams</code>	3	1–8 команд
<code>--cars</code>	2	1–4 машины в команде
<code>--lanes</code>	4	1–6 дорожек
<code>--length</code>	24	3–120 клеток; больше общего числа машин
<code>--laps</code>	2	1–100 кругов
<code>--rounds</code>	100	1–10000 раундов
<code>--seed</code>	1	0–4294967295 в текущем окружении
<code>--strategy</code>	<code>greedy</code>	<code>greedy</code> или <code>random</code>
<code>--delay</code>	120	0–5000 мс

Примеры запуска:

```
./race --delay 300  
./race --no-animation --strategy random --seed 7  
./race --quiet --rounds 1 --log race.log  
./race --teams 8 --cars 4 --length 33 --lanes 2 --strategy random --seed 0
```

В интерактивном терминале по умолчанию включена анимация: после каждого перехода на соседнюю клетку экран обновляется и выдерживается пауза 120 мс. Клетки отображаются с границами, машины — номерами и цветами команд; текущая машина выделяется. Видны шаг маршрута, стоимость, дорожка, клетка, круги, порядок участников, выбранный маршрут и текущее событие. В обозначении маршрута В — вперёд, И — внутрь, Н — наружу. Блокирование, нулевой ход и фактический сход показаны отдельными кадрами. Параметр `--delay MS` задаёт паузу от 0 до 5000 мс. Для длинной трассы отображается участок около текущей машины по ширине терминала.

`--no-animation` и `--delay 0` включают обычный текстовый вывод. Анимация также отключается при `--quiet`, перенаправлении `stdout` и `TERM=dumb`. Управляющие коды терминала не попадают в журнал. После гонки и обработанного прерывания экран и курсор восстанавливаются.

Вывод отражает стартовую расстановку, порядок хода, его предел, блокирование переходов, маршрут, смену дорожки, стоимость, завершённые круги, нулевой ход, сход, финиш и текущее положение машин. Событие премии отсутствует, поскольку премии отключены.

`--quiet` оставляет на экране параметры и итоговые таблицы. `--log FILE` сохраняет полный текстовый журнал событий независимо от `--quiet` и анимации. Файл журнала открывается для перезаписи; ошибки открытия и записи приводят к ненулевому коду завершения.

При `SIGINT` (Ctrl+C) или `SIGTERM` обработчик только устанавливает флаг. Текущий раунд заканчивается, выполняется уборка, формируются результаты и закрывается журнал. Код завершения — 130 для `SIGINT` и 143 для `SIGTERM`. При прерывании задержки анимации отменяются, поэтому завершение текущего раунда не ожидает оставшихся пауз. Обычное завершение, включая достижение лимита раундов, возвращает 0; ошибки параметров и ввода-вывода — 1.

6 Проверка и фактические результаты

Команда `make test` выполняет проверки на C и Python. Для них нужен Python 3; сама программа использует C, `libc` и системные интерфейсы Linux/POSIX для сигналов, задержек и определения размера терминала.

Инвариант	Обеспечение в программе
В клетке не более одной машины	Маршрут допускает только свободные клетки; занятость обновляется на каждом переходе
Нет движения назад и по диагонали	Допустимы только вперёд, внутрь и наружу
Стоимость не превышает предел	Каждый шаг проверяется по оставшемуся бюджету
Порядок пересчитывается после раунда	Массив фиксирован внутри раунда; после уборки создаётся заново
Выбывшие больше не ходят	Порядок включает только состояние ACTIVE

- Проверены жеребьёвка, стоимость боковых переходов, блокирование, запрет повторного посещения, точный финиш, отложенное снятие и ничьи.
- На C проверены 200 гонок двух стратегий: каждый маршрут и состояние занятости после каждого хода и конца раунда.
- Дополнительно проверены 3248 маршрутов: все расстановки на трассе 2×4 с одной движущейся машиной и до двух препятствий, обе стратегии, пределы 1–7. Вызовы самого `run_round` проверяют пересчёт порядка после раунда и неизменность состояния ранее финишировавших и сошедших.
- На Python независимо восстановлены 28 журналов: проверены переходы, стоимость, неизменность порядка внутри раунда, следующий предел хода, круги, финиш, сход и итоговые классификации.
- Проверены неверные и предельные параметры, воспроизводимость, полный журнал при `--quiet`, ошибка записи, SIGINT и SIGTERM.
- В псевдотерминале проверены отдельные кадры после каждого перехода, совпадение координат с клетками, видимые паузы, цвета и границы, текстовый журнал, ширина экрана и восстановление терминала при сигналах. Отдельная плотная гонка проверяет видимость блокирования, нулевого хода, финиша и всех 22 сходов; в кадре схода машина уже отсутствует на трассе.
- Сборка выполнена с `-std=c11 -Wall -Wextra -Wpedantic -Werror`. Дополнительный анализ GCC -fanalyzer не сообщил ошибок.
- ASan/UBSan не запущены: в окружении отсутствуют их динамические и статические библиотеки.

Результаты ниже получены в текущем окружении. Генератор `rand()` зависит от реализации `libc`; одинаковый `seed` воспроизводит гонку в одной и той же сборке и окружении, но не гарантирует совпадение между разными платформами.

Пример	Параметры сверх стандартных	Раундов	Результат
Обычная гонка	нет	11	Все 6 машин финишировали; порядок: 2, 4, 5, 6, 1, 3; команды: 1, 2, 3
Лимит	<code>--rounds 1</code>	1	Все машины не завершили; порядок: 1, 5, 3, 6, 2, 4

Пример	Параметры сверх стандартных	Раундов	Результат
Случайная стратегия	<code>--strategy random --seed 7</code>	24	Все финишировали; победила машина 6, команда 3
Блокировки и сход	<code>--teams 8 --cars 4 --length 33 --lanes 2 --laps 2 --strategy random --seed 0</code>	30	10 машин финишировали, 22 сошли; победила машина 29, команда 8

Полные журналы этих запусков сохранены в `examples/default.log`, `examples/limit.log`, `examples/random.log` и `examples/blocked.log`.

Фрагмент итогов обычной гонки:

Место	Машина	Команда	Продвижение	Состояние
1	2	1	48	финиш
2	4	2	48	финиш
3	5	3	48	финиш
4	6	3	48	финиш
5	1	1	48	финиш
6	3	2	48	финиш

7 Использование ИИ и ограничения

При разработке проекта я использовал Codex для работы с кодом, тестами и документацией. Промпты и критическая оценка собраны в `AI_PROMPTS.md`.

В программе разделены планирование и выполнение маршрута, продвижение и стоимость хода. Порядок участников фиксируется на раунд, а снятие машин выполняется после его завершения. Независимые проверки маршрутов и журналов подтверждают согласованность этих решений с правилами гонки в проверенных сценариях.

При сверке с документами обнаружен и исправлен недостаток анимации: часть обязательных событий ранее была доступна только в обычном текстовом режиме или журнале. Теперь порядок и маршрут показаны в кадрах, а блокирование и фактический сход имеют отдельные сообщения. Порядок следующего раунда явно пересчитывается сразу после уборки. Подробная сверка пяти инвариантов и требований приведена в `AUDIT.md`.

Геометрия трассы ограничена кольцом с одинаковым числом клеток на дорожках. Повороты из рисунка, произвольные графы трасс и переменная ширина не поддерживаются. Стратегии локальные, без командной тактики и оптимального планирования. Дополнительные премии и снятие отставших не включены. Наглядность обеспечивается текстовыми таблицами и анимацией развёрнутой трассы с регулируемой задержкой. Графического интерфейса нет.

Приложение Промпты по этапам разработки

1 Разбор задания

Я разобрал условие варианта 20 «Автогонки» и хочу проверить, что ничего не пропустил. Сверь правила движения, порядок раундов и завершение гонки с обоими PDF. Отдельно уточни, какие дополнительные правила можно оставить отключёнными.

2 Трасса

Для трассы я выбираю двумерный массив: дорожка и клетка. После последней клетки будет первая, а боковой переход сохранит сектор. Подойдёт ли это для задания? Помоги точно описать упрощение геометрии, чтобы оно было понятно из отчёта.

3 Продвижение машины

В Car у меня отдельно хранятся положение на трассе и общее продвижение. Стоимость маршрута считаю отдельно: наружу стоит 3, хотя расстояние вперёд не меняется. Проверь, правильно ли я разделил эти величины и как по ним считать круги.

4 Структуры программы

Я набросал Config, Car, Route и Race и разделил программу на функции для старта, выбора маршрута, движения и конца раунда. Посмотри, хватает ли полей и нет ли лишнего дублирования. Особенно проверь связь координат машины с массивом занятости.

5 Стартовая расстановка

В initialize_race я ставлю машины на внешнюю дорожку перед линией и задаю продвижение -1, -2 и дальше. Сначала идут первые машины команд, затем вторые. Помоги добавить жеребьёвку команд по seed и проверь, чтобы стартовые клетки не повторялись.

6 Порядок ходов

В create_order я сортирую активные машины по продвижению, затем по дорожке и номеру. Посмотри, правильно ли работают равные значения. Ещё проверь расчёт бюджета в run_round: 4 у лидера, предыдущая фактическая стоимость плюс 2 у второго и плюс 1 у остальных.

7 Допустимый маршрут

Пишу choose_route отдельно от выполнения движения. Проверяю границы, занятость, остаток бюджета и посещённые клетки. Нужно ли отмечать исходную клетку как посещённую? Проверь, что машина не сможет за один ход вернуться в неё после обхода кольца.

8 Стратегии

В `greedy` я выбираю сначала вперёд, потом внутрь, потом наружу. Помоги добавить `random`, чтобы он выбирал только допустимые переходы. Проверь случай, когда продолжить путь нельзя, но машина уже сделала несколько шагов: такой ход не должен считаться нулевым.

9 Движение и круги

В `execute_route` я обновляю занятость после каждого перехода и увеличиваю продвижение только при движении вперёд. Посмотри расчёт кругов и финиша. Где лучше обрывать маршрут, чтобы машина остановилась точно на линии, даже если остался бюджет?

10 Сход и снятие с трассы

Снятие машин я вынес в `finish_round`. До конца раунда машина с нулевым ходом остаётся препятствием, как и финишировавшая. Проверь этот порядок и случай, когда другую машину позднее поставили в клетку ранее снятого участника: повторная уборка не должна её удалить.

11 Итоговые места

В результатах я ставлю финишировавших по очереди финиша, остальных — по продвижению. Для команд хочу учитывать лучшую машину. Посмотри, согласованы ли правила ничьих, и помоги проверить завершение по лимиту: оставшиеся машины должны быть не завершившими гонку, а не сошедшими.

12 Параметры и журнал

Я добавил разбор аргументов и проверку диапазонов до запуска гонки. Проверь неверные числа и несовместимые размеры. Помоги доработать вывод в журнал: он должен сохранять все события при `--quiet` и обнаруживать ошибки записи, в том числе при закрытии файла.

13 Прерывание

Хочу добавить `Ctrl+C` и `SIGTERM`, чтобы сохранялись результаты. Подскажи, как правильно сделать обработчик сигнала: думаю, в нём нужен только флаг, а печать и закрытие файла должны остаться в основном коде. Помоги проверить завершение текущего раунда и отмену задержек.

14 Анимация

Теперь хочу добавить к текстовому выводу анимацию по клеткам. Помоги сделать перерисовку после каждого перехода, границы клеток, цвета команд и выделение текущей машины. На экране должны оставаться порядок, маршрут и события. Журнал должен быть обычным текстом, а экран и курсор после гонки нужно восстановить.

15 Проверка

Посмотри готовый код и проверь все пять инвариантов из условия. Помоги сделать независимые тесты, которые заново считают стоимость, координаты и занятость, а не повторяют функции программы. Отдельно проверь блокировки, финиш, отложенный сход, сигналы и настоящие кадры анимации.

16 Отчёт

Помоги оформить отчёт по текущему коду и результатам тестов. Опиши модель, функции, способы запуска и завершения, добавь примеры гонок. Проверь, чтобы описание совпадало с реализацией, и укажи найденные исправления. ФИО и группу оставь для заполнения.